

رسالة محمد



یادگیری ماشین

مدرس: محمدرضا محمدی

زمستان ۱۴۰۱

ماشین‌های بردار پشتیبان

Support Vector Machines

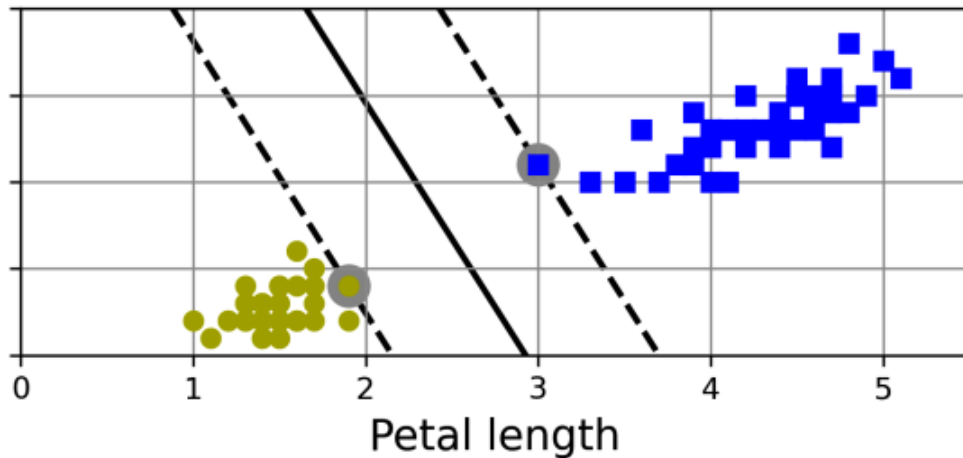
SVM خطی برای دسته‌بندی

$$\boldsymbol{\theta}^T \mathbf{x} = \theta_0 x_0 + \cdots + \theta_n x_n = \mathbf{w}^T \mathbf{x} + b = w_1 x_1 + \cdots + w_n x_n + b$$

$$\hat{y} = \begin{cases} 0 & \text{if } \boldsymbol{\theta}^T \mathbf{x} < 0.5 \\ 1 & \text{if } \boldsymbol{\theta}^T \mathbf{x} \geq 0.5 \end{cases}$$

• چگونه وزن‌های مربوط به خط دارای بیشترین حاشیه را بدست بیاوریم؟

- حاشیه را جایی تعیین می‌کنیم که $\boldsymbol{\theta}^T \mathbf{x} = \pm 1$ شود
- بیشترین حاشیه زمانی بدست می‌آید که $\mathbf{w}$ کمترین مقدار را داشته باشد
- در حاشیه سخت، هیچکدام از نمونه‌ها نباید از حاشیه عبور کنند
- مسئله بهینه‌سازی مقید زیر را باید حل کنیم:



$$\underset{\mathbf{w}, b}{\text{minimize}} \quad \frac{1}{2} \mathbf{w}^T \mathbf{w}$$

$$\text{subject to} \quad t^{(i)} (\mathbf{w}^T \mathbf{x}^{(i)} + b) \geq 1 \quad \text{for } i = 1, 2, \dots, m$$

$$t^{(i)} = \begin{cases} -1 & \text{if } y^{(i)} = 0 \\ +1 & \text{if } y^{(i)} = 1 \end{cases}$$

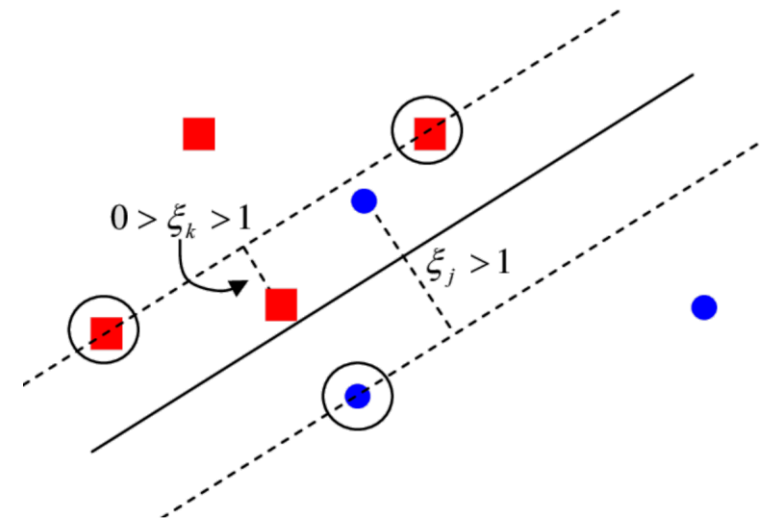
حاشیه نرم

$$\begin{aligned} & \underset{\mathbf{w}, b}{\text{minimize}} && \frac{1}{2} \mathbf{w}^\top \mathbf{w} \\ & \text{subject to} && t^{(i)} (\mathbf{w}^\top \mathbf{x}^{(i)} + b) \geq 1 \quad \text{for } i = 1, 2, \dots, m \end{aligned}$$

• در حاشیه نرم، اجازه داده می شود تا برخی نمونه ها بتوانند حاشیه را بشکنند

- متناسب با شکستن حاشیه، به تابع هزینه افزوده می شود

$$\begin{aligned} & \underset{\mathbf{w}, b, \zeta}{\text{minimize}} && \frac{1}{2} \mathbf{w}^\top \mathbf{w} + C \sum_{i=1}^m \zeta^{(i)} \\ & \text{subject to} && t^{(i)} (\mathbf{w}^\top \mathbf{x}^{(i)} + b) \geq 1 - \zeta^{(i)} \quad \text{and} \quad \zeta^{(i)} \geq 0 \quad \text{for } i = 1, 2, \dots, m \end{aligned}$$



حاشیه نرم

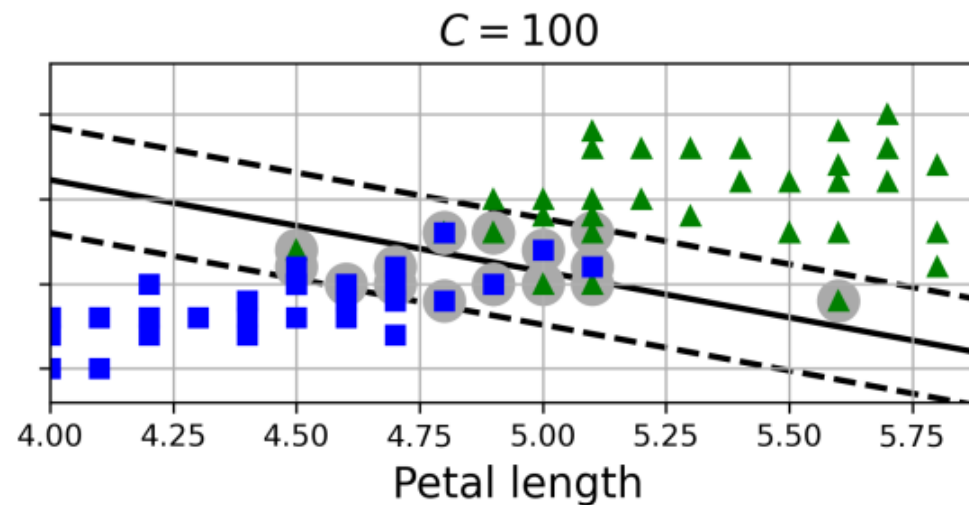
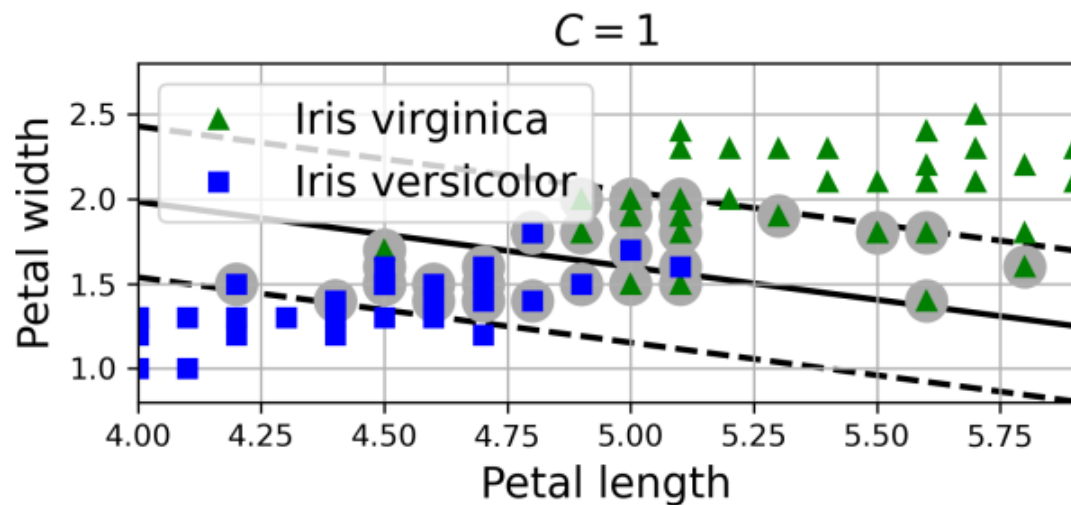
```
from sklearn.svm import LinearSVC
```

```
iris = load_iris(as_frame=True)  
X = iris.data[["petal length (cm)", "petal width (cm)"]].values  
y = (iris.target == 2) # Iris virginica
```

```
svm_clf = make_pipeline(StandardScaler(),  
                        LinearSVC(C=1, random_state=42))
```

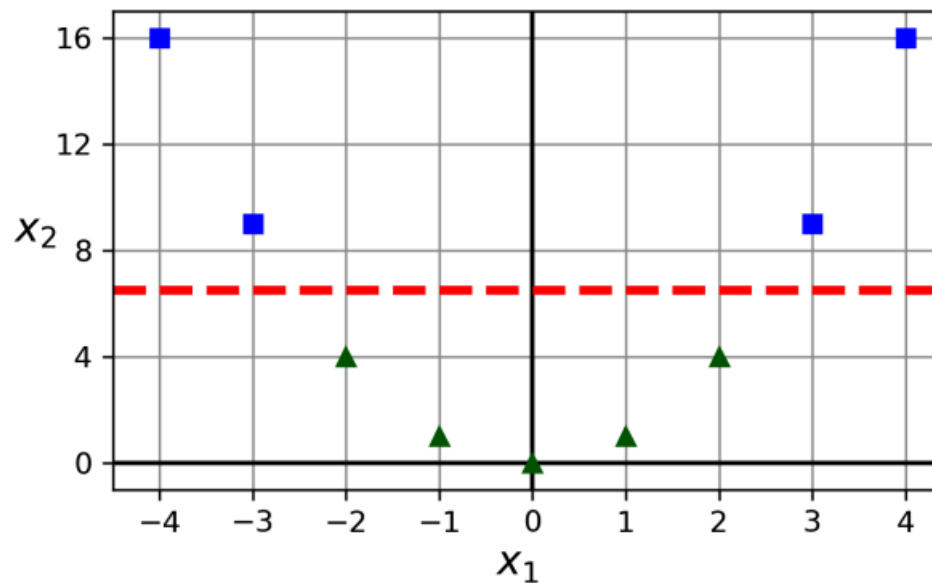
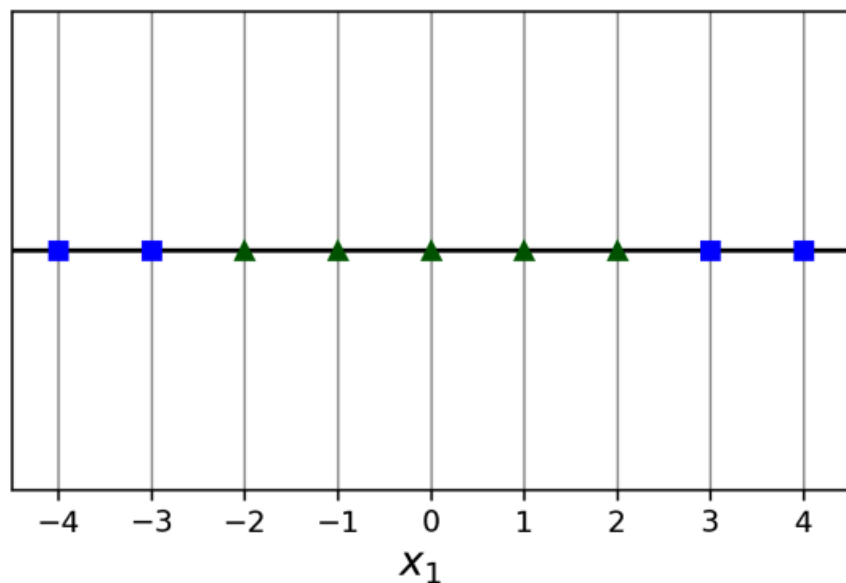
```
svm_clf.fit(X, y)
```

- مدل LinearSVC احتمال محاسبه نمی‌کند بلکه فاصله تا خط تصمیم را محاسبه کرده و بر اساس آن پیش‌بینی می‌کند



SVM غیرخطی برای دسته‌بندی

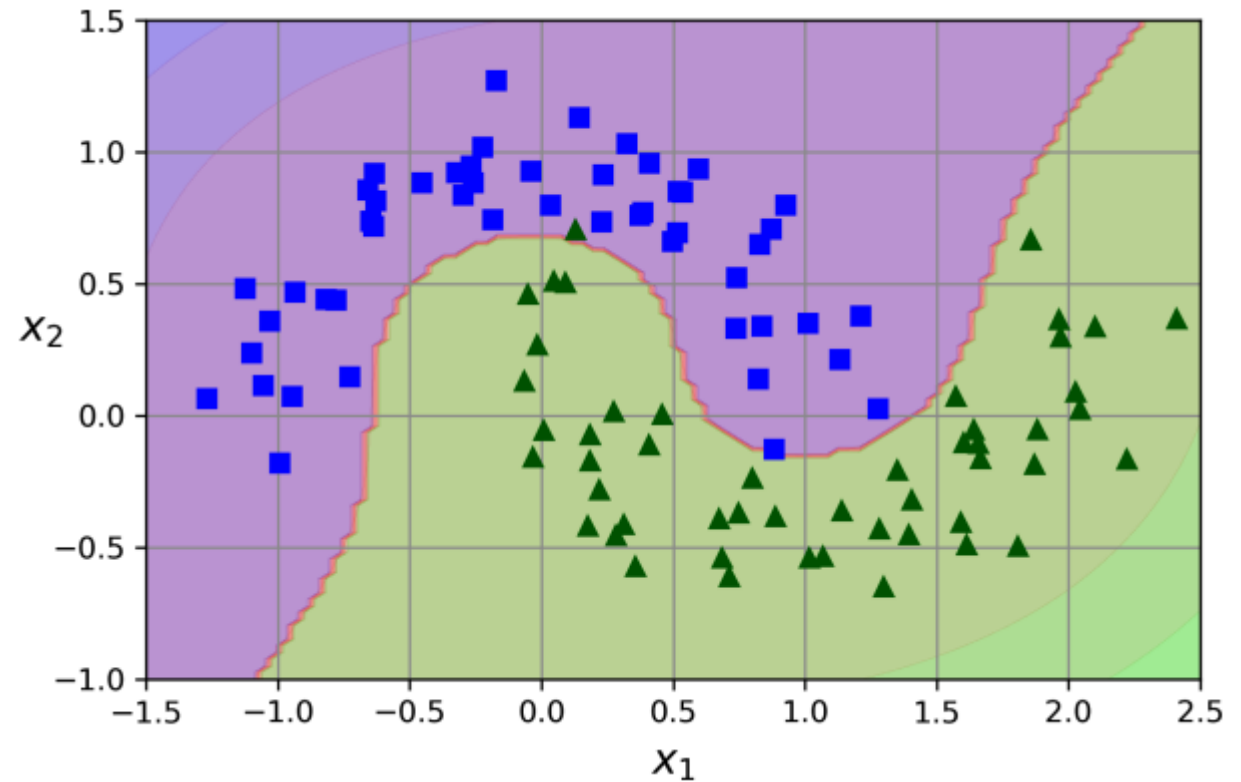
- برای بسیاری از مجموعه داده‌ها، مدل خطی نمی‌تواند کارایی لازم را داشته باشد
 - یک رویکرد افزودن ویژگی‌های غیرخطی (مانند چند جمله‌ای) و استفاده از مدل خطی است
- $x_2 = (x_1)^2$ -



```
from sklearn.datasets import make_moons
from sklearn.preprocessing import PolynomialFeatures
```

```
X, y = make_moons(n_samples=100, noise=0.15, random_state=42)
```

```
polynomial_svm_clf = make_pipeline(
    PolynomialFeatures(degree=3),
    StandardScaler(),
    LinearSVC(C=10, max_iter=10_000, random_state=42)
)
polynomial_svm_clf.fit(X, y)
```



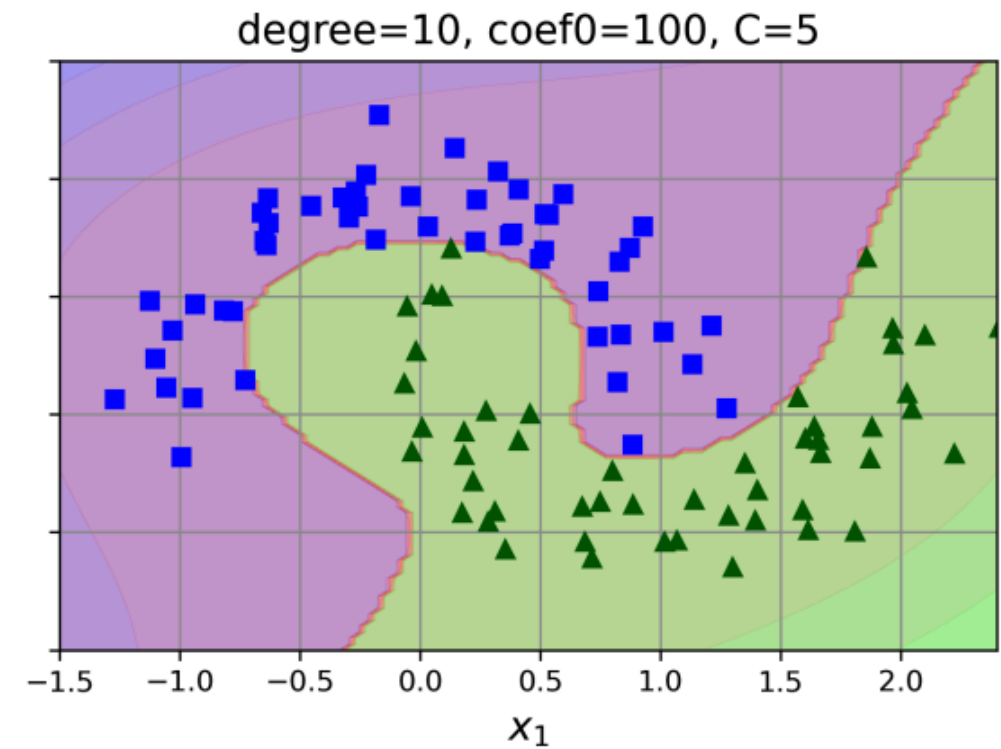
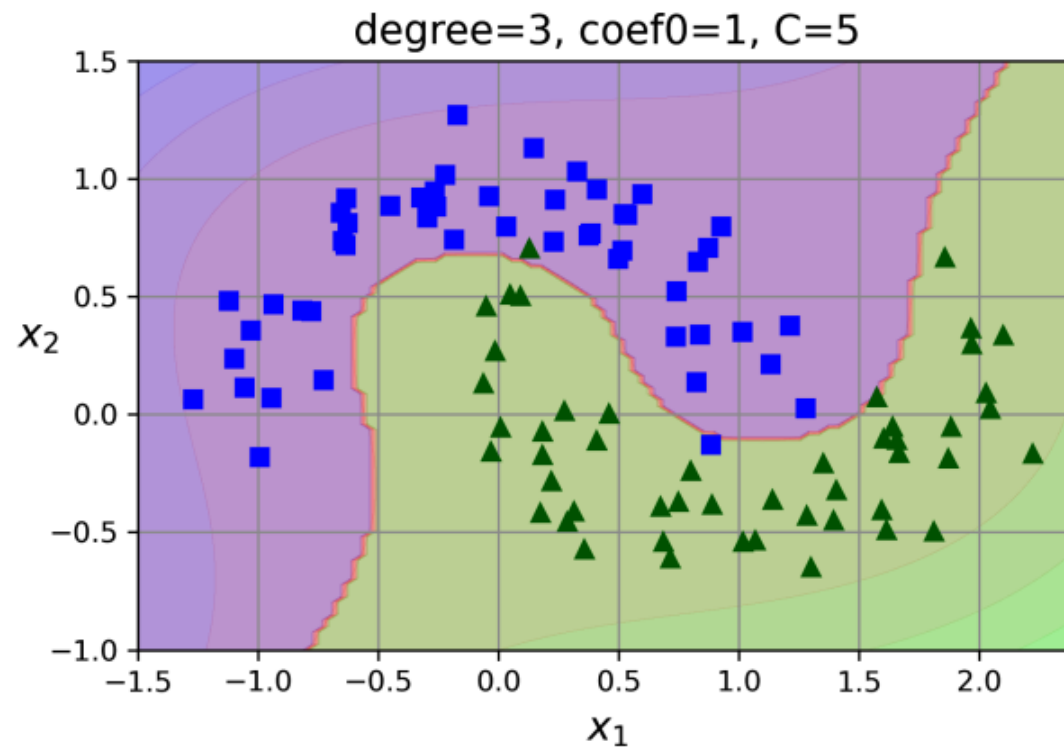
کرنل چند جمله‌ای

- افزودن ویژگی‌های چند جمله‌ای به سادگی قابل پیاده‌سازی است و برای تمام الگوریتم‌های یادگیری ماشین قابل استفاده است
 - اگر درجه چند جمله‌ای کم باشد، نمی‌تواند مجموعه داده‌های بسیار پیچیده را به خوبی مدل کند
 - اگر درجه چند جمله‌ای زیاد باشد، تعداد بسیار زیادی ویژگی باید ساخته شود که مدل را بسیار کند می‌کند
- خوشبختانه در SVM با استفاده از یک تکنیک ریاضی به نام kernel trick، این امکان فراهم می‌شود که نتیجه مشابه با استفاده از ویژگی‌های چند جمله‌ای حتی با درجه بسیار بالا را بدست آورد، بدون اینکه واقعاً آنها را اضافه کرد

```
from sklearn.svm import SVC
```

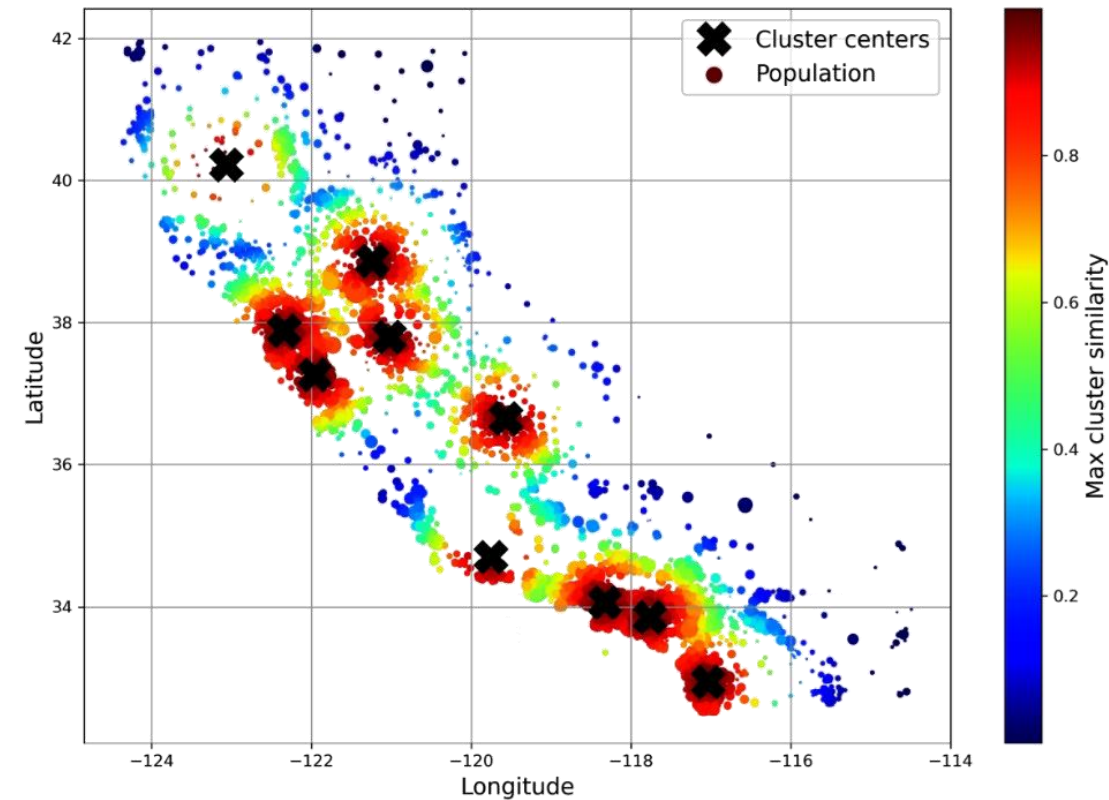
```
poly_kernel_svm_clf = make_pipeline(StandardScaler(),  
                                     SVC(kernel="poly", degree=3, coef0=1, C=5))
```

```
poly_kernel_svm_clf.fit(X, y)
```



ویژگی‌های مشابهت

- یک راه دیگر برای کار با مسائل غیرخطی، افزودن ویژگی‌هایی است که مشابهت نمونه‌ها با نقاط landmark را محاسبه می‌کنند



ویژگی‌های مشابهت

- یک راه دیگر برای کار با مسائل غیرخطی، افزودن ویژگی‌هایی است که مشابهت نمونه‌ها با نقاط landmark را محاسبه می‌کنند

$$x_2 = \exp(-\gamma(x_1 + 2)^2)$$

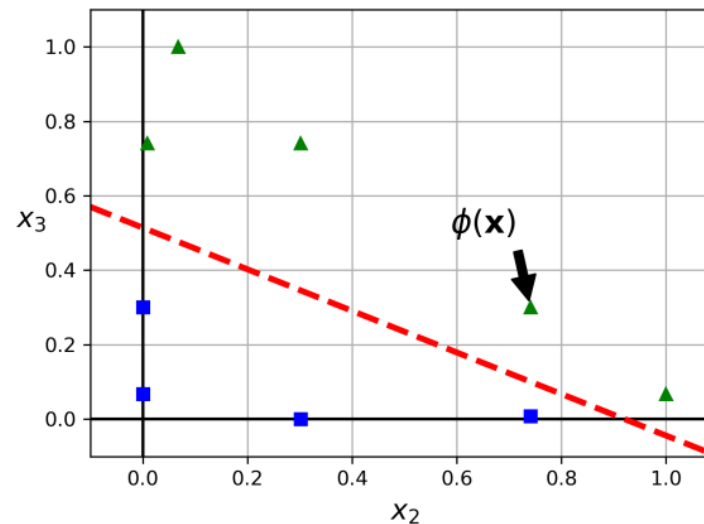
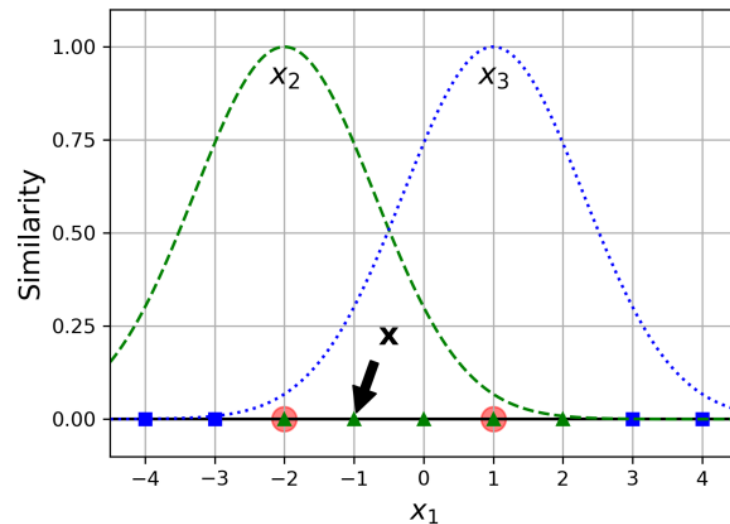
$$x_3 = \exp(-\gamma(x_1 - 1)^2)$$

$$\gamma = 0.3$$

- چطور landmarkها را انتخاب کنیم؟

▪ می‌توان به ازای هر نمونه یک landmark تعریف کرد

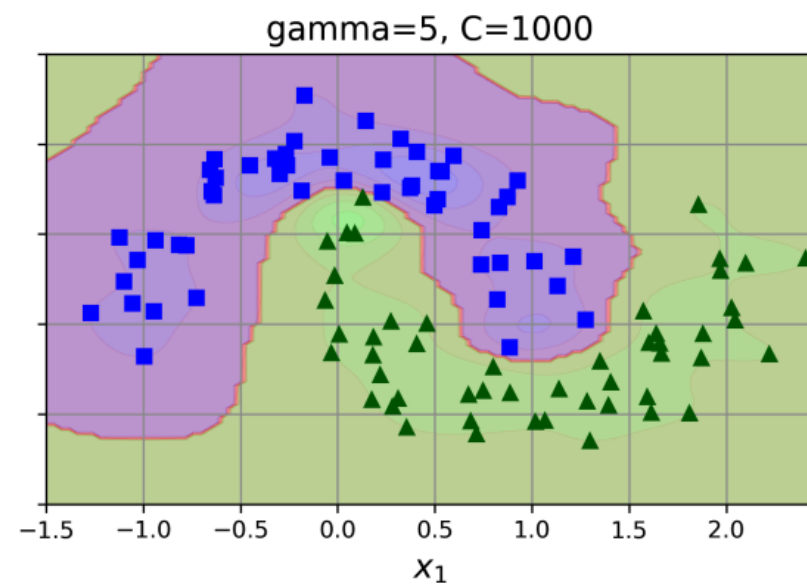
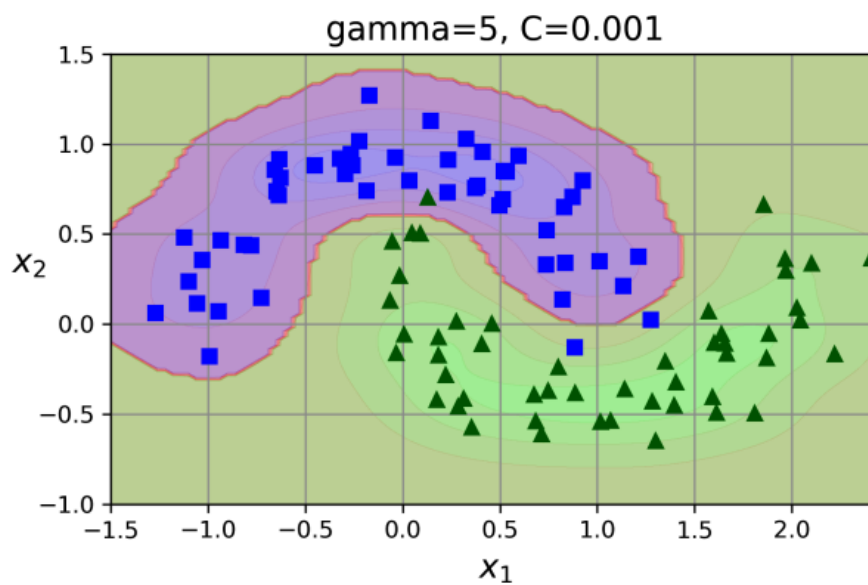
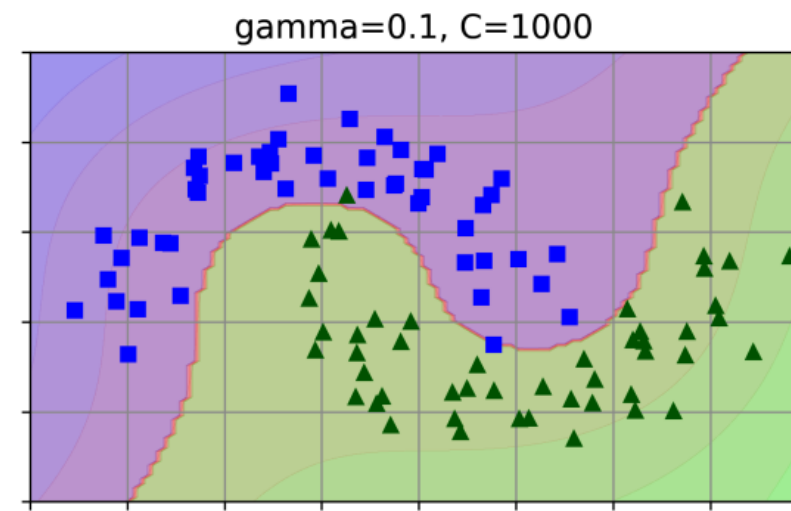
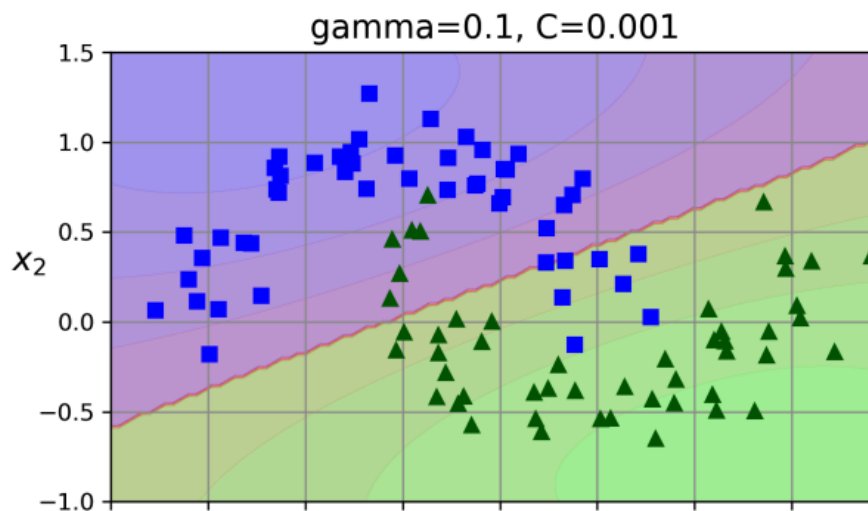
▪ برای مجموعه داده‌های بزرگ، ابعاد ویژگی‌ها خیلی زیاد می‌شود



Gaussian RBF Kernel

- ویژگی‌های مشابهت می‌توانند توسط هر الگوریتم یادگیری ماشین استفاده شوند اما ممکن است از لحاظ هزینه محاسباتی به صرفه نباشند
- خوشبختانه SVM با استفاده از kernel trick می‌تواند نتایجی مشابه با استفاده از ویژگی‌های مشابهت بسیار زیاد نتیجه بدهد بدون آنکه واقعا آنها را محاسبه کند

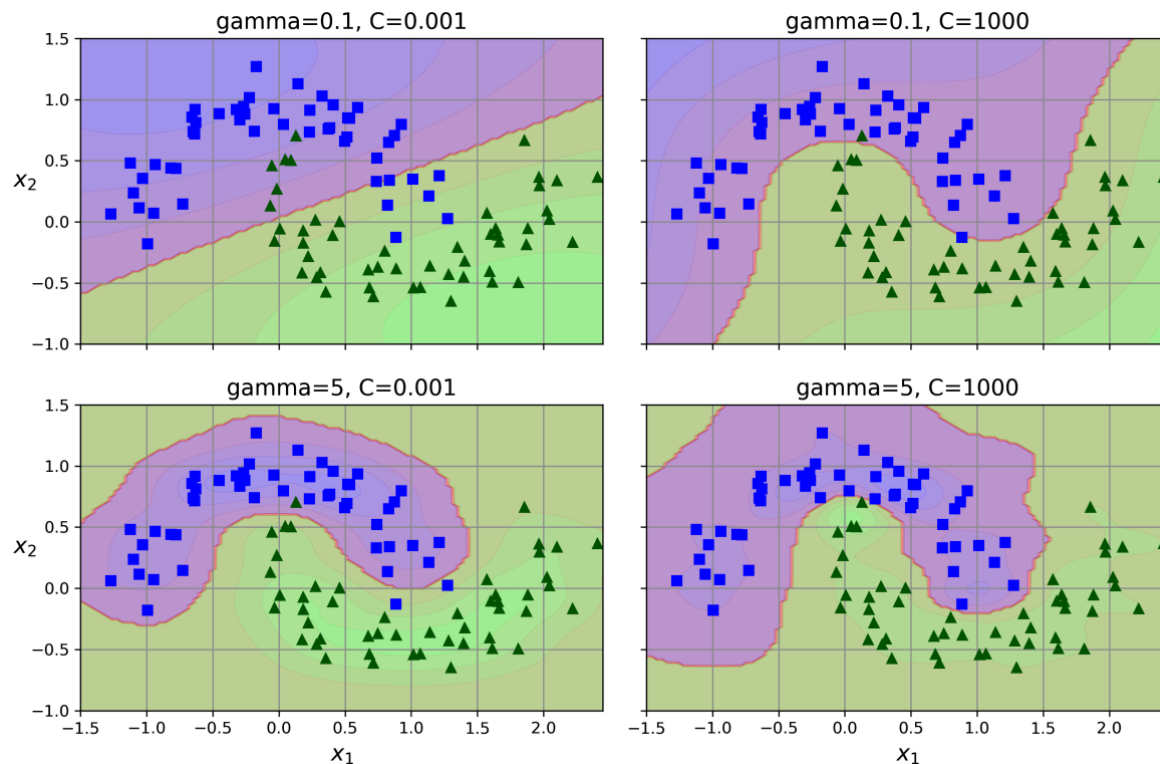
```
rbf_kernel_svm_clf = make_pipeline(StandardScaler(),  
                                    SVC(kernel="rbf", gamma=5, C=0.001))  
rbf_kernel_svm_clf.fit(X, y)
```



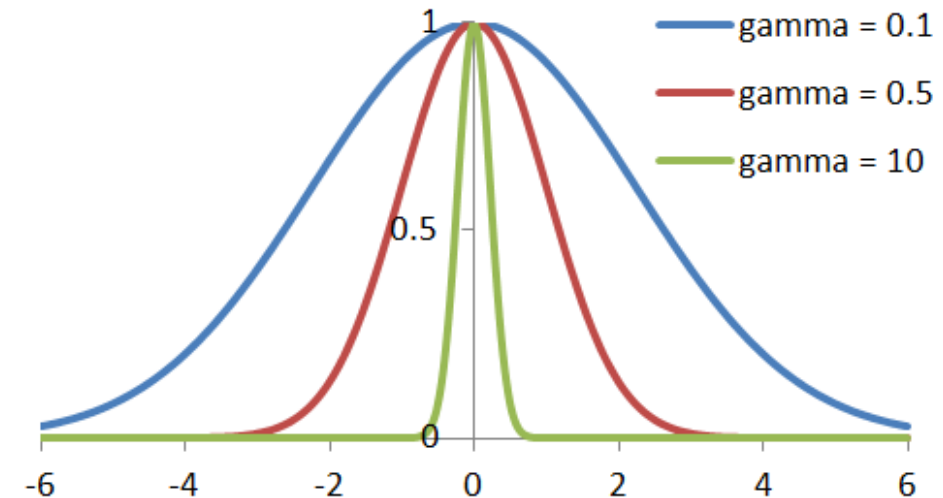
Gaussian RBF Kernel

- افزایش پارامتر γ باعث می‌شود هر نمونه اثرگذاری محدودتری داشته باشد و مرز تصمیم نامنظم‌تر شود
- کاهش پارامتر γ باعث می‌شود هر نمونه اثرگذاری گسترده‌تری داشته باشد و مرز تصمیم هموارتر شود

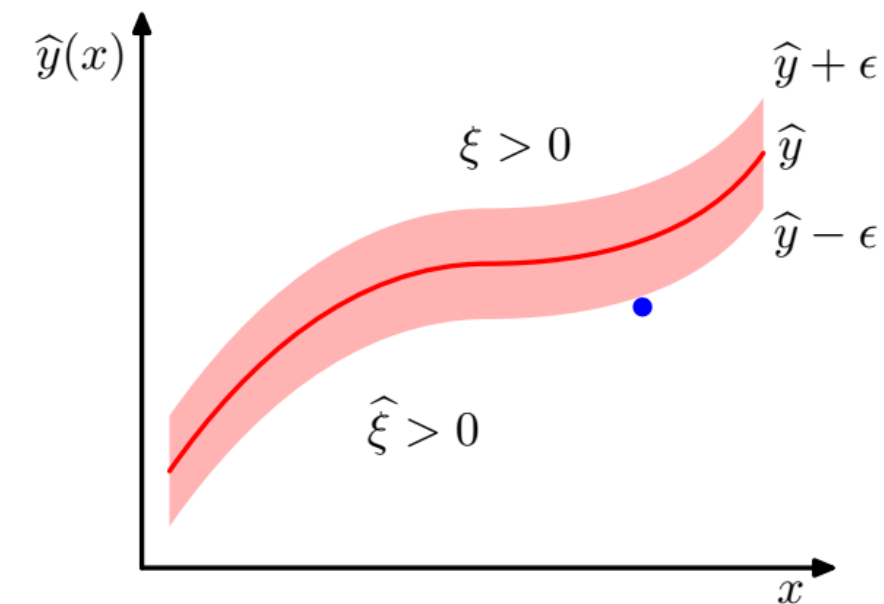
- γ به نوعی نقش منظم‌سازی دارد



$$\exp(-\gamma \|x - x_l\|^2)$$



SVM خطی برای رگرسیون



- SVR تلاش می‌کند تابعی را بدست بیاورد که اکثر نمونه‌های آموزش در یک حاشیه کوچک در اطراف آن باشند

• تابع ضرر MSE منظم‌شده:

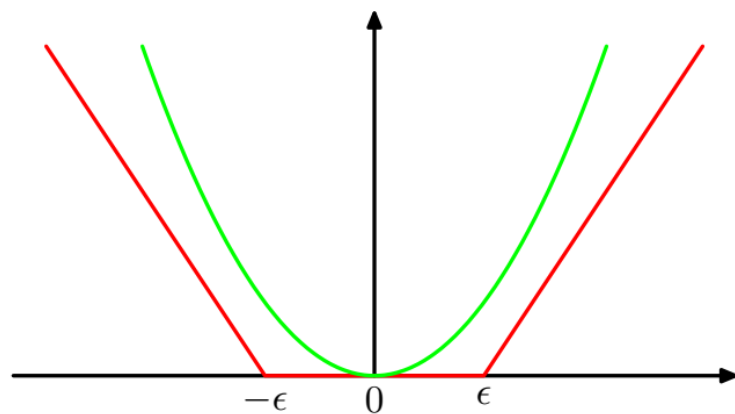
$$\frac{1}{m} \sum_{i=1}^m (\hat{y}^{(i)} - y^{(i)})^2 + \frac{\lambda}{2} \|\mathbf{w}\|^2$$

- در SVR، نمونه‌ها باید در حاشیه‌ای کوچک قرار داشته باشند و اگر خارج از حاشیه باشند جریمه می‌شوند

$$y^{(i)} \leq \hat{y}^{(i)} + \epsilon + \xi^{(i)}$$

$$y^{(i)} \geq \hat{y}^{(i)} - \epsilon - \hat{\xi}^{(i)}$$

$$C \sum_{i=1}^m (\xi^{(i)} + \hat{\xi}^{(i)})^2 + \frac{\lambda}{2} \|\mathbf{w}\|^2$$

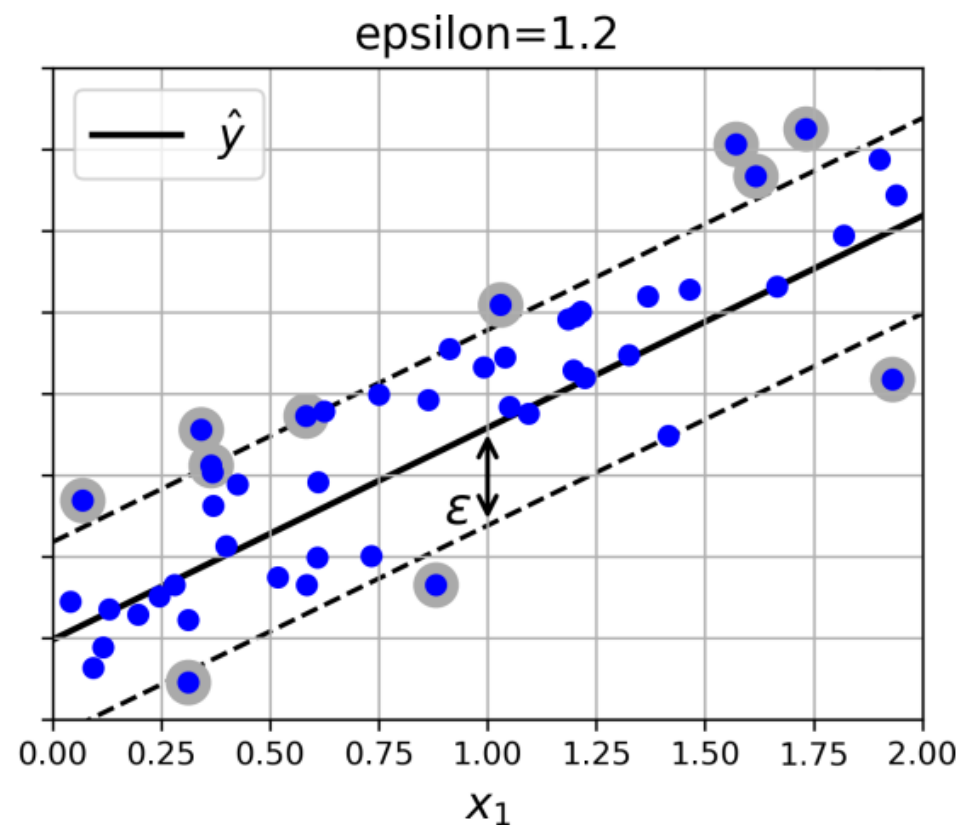
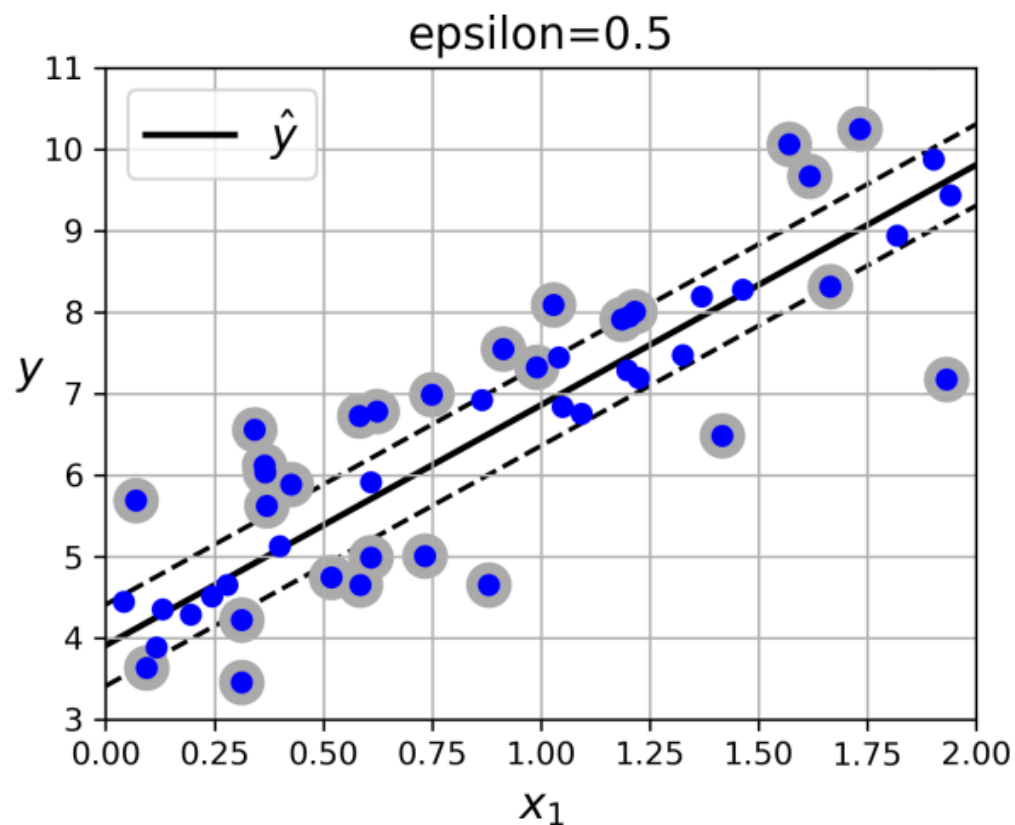


```
from sklearn.svm import LinearSVR
```

```
X, y = [...] # a linear dataset
```

```
svm_reg = make_pipeline(StandardScaler(),  
                        LinearSVR(epsilon=0.5, random_state=42))
```

```
svm_reg.fit(X, y)
```



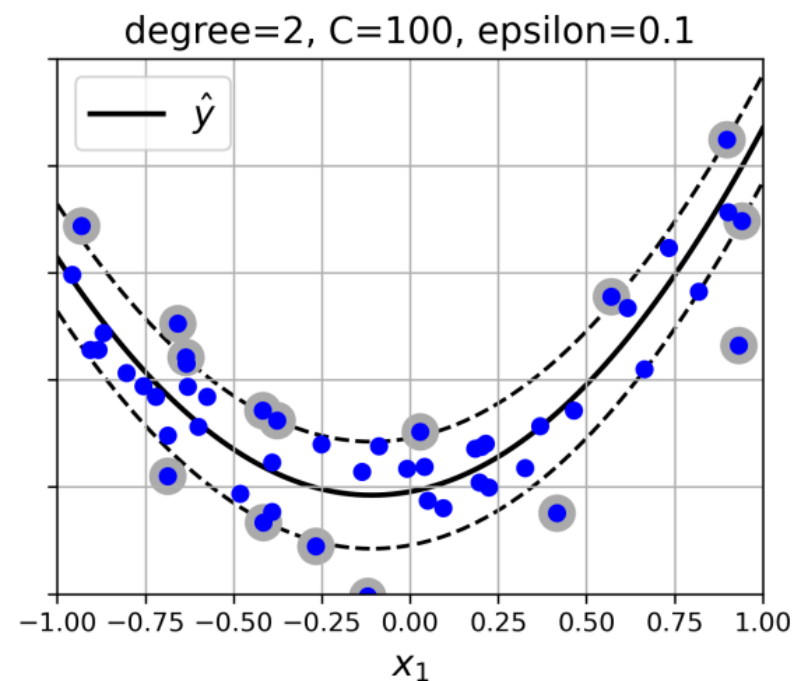
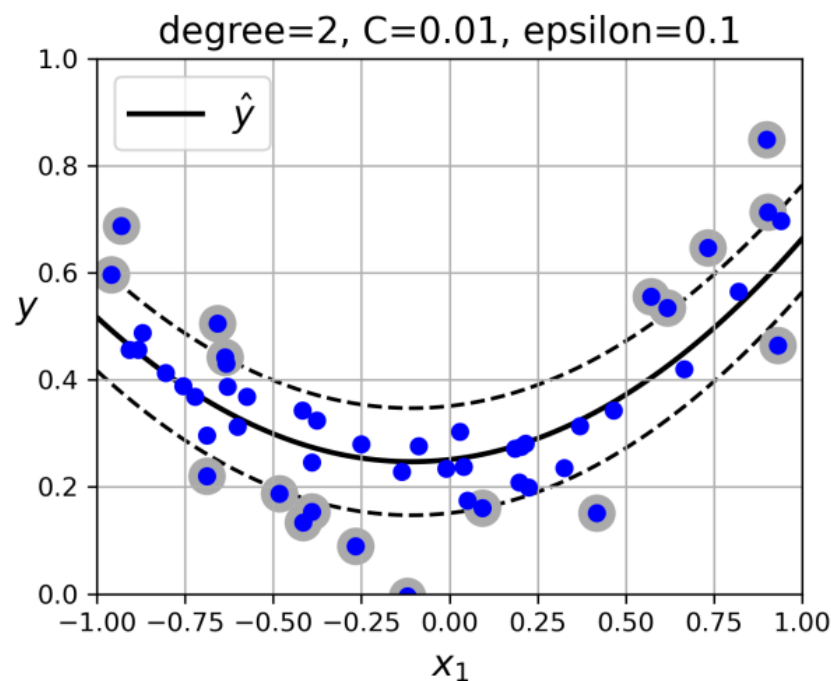
SVR غیر خطی

- برای رگرسیون غیر خطی می توان از ویژگی های غیر خطی استفاده کرد که در SVR با استفاده از kernel trick به خوبی قابل انجام است

```
X, y = [...] # a quadratic dataset
```

```
svm_poly_reg = make_pipeline(StandardScaler(),  
                              SVR(kernel="poly", degree=2, C=0.01, epsilon=0.1))
```

```
svm_poly_reg.fit(X, y)
```

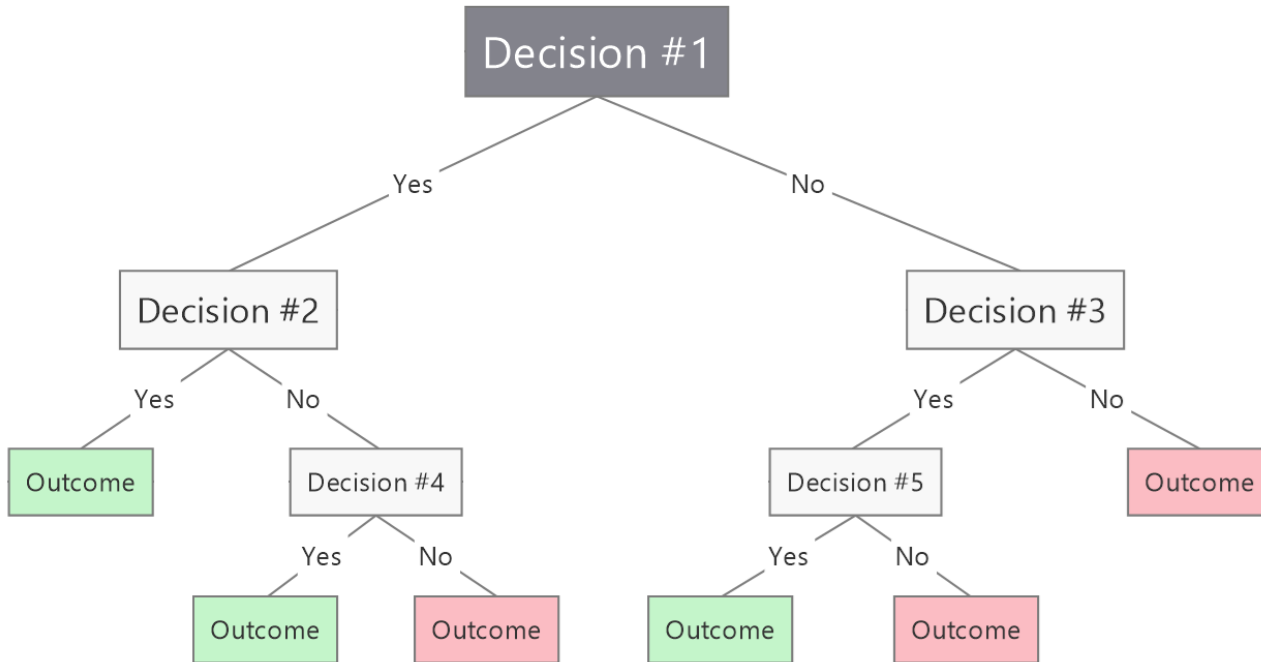


درخت تصمیم

Decision Trees

درخت تصمیم

- DT یک الگوریتم یادگیری ماشین قدرتمند است که می‌تواند برای مسائل مختلف از جمله دسته‌بندی و رگرسیون استفاده شود



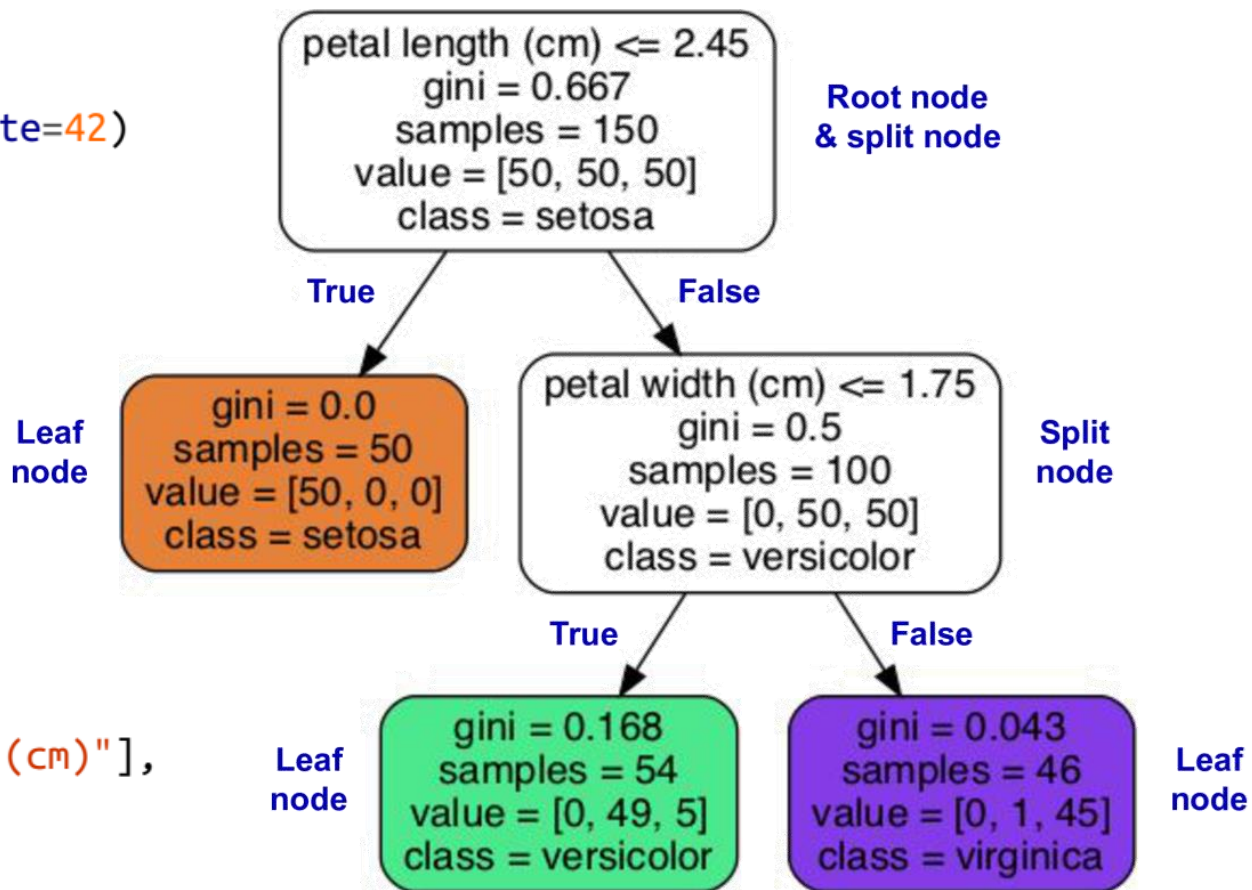
```
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier
```

```
iris = load_iris(as_frame=True)
X_iris = iris.data[["petal length (cm)", "petal width (cm)"]].values
y_iris = iris.target
```

```
tree_clf = DecisionTreeClassifier(max_depth=2, random_state=42)
tree_clf.fit(X_iris, y_iris)
```

```
from sklearn.tree import export_graphviz
```

```
export_graphviz(
    tree_clf,
    out_file="iris_tree.dot",
    feature_names=["petal length (cm)", "petal width (cm)"],
    class_names=iris.target_names,
    rounded=True,
    filled=True
)
```



انجام پیش‌بینی

• مثال:

petal width = 0.9 cm, petal length = 2.1 cm -

▪ setosa

petal width = 0.9 cm, petal length = 2.5 cm -

▪ versicolor

petal width = 1.9 cm, petal length = 2.5 cm -

▪ virginica

• آیا در آموزش DT، نرمال‌سازی یا تغییر مقیاس

ویژگی‌ها لازم است؟

- خیر!

